

Rapport Interconnexion

Kraifi Ilian

Krief Benjamin

Mediani Hocine

Mbaitodjimreou Eldjimbaye Bonheur

Berrad Marouane

Nadi Aymen

Sommaire :

1. Principaux choix et implémentation

- i. Serveur SMTP
- ii. VLAN dans l'entreprise
- iii. VPN entre l'entreprise et les particuliers
- iv. Les serveurs WEB (entreprise et public)
- v. Les serveurs DNS (entreprise et public)
- vi. DHCP client / accès de type "box-like"
- vii. Serveur VOIP
- viii. Routage dynamique par OSPF
- ix. DHCP AS (accès box-like et gestion des souscriptions)
- x. DNS split horizon au niveau des box clients
- xi. Mise en place des réseaux privés et du firewall

2. Répartition des tâches

1. Principaux choix et implémentation

Le projet d'interconnexion a pour but d'implémenter un AS (Autonomous System) représentant un FAI (Fournisseur d'Accès Internet). Ce dernier se doit alors d'avoir quelques services de base, tels que :

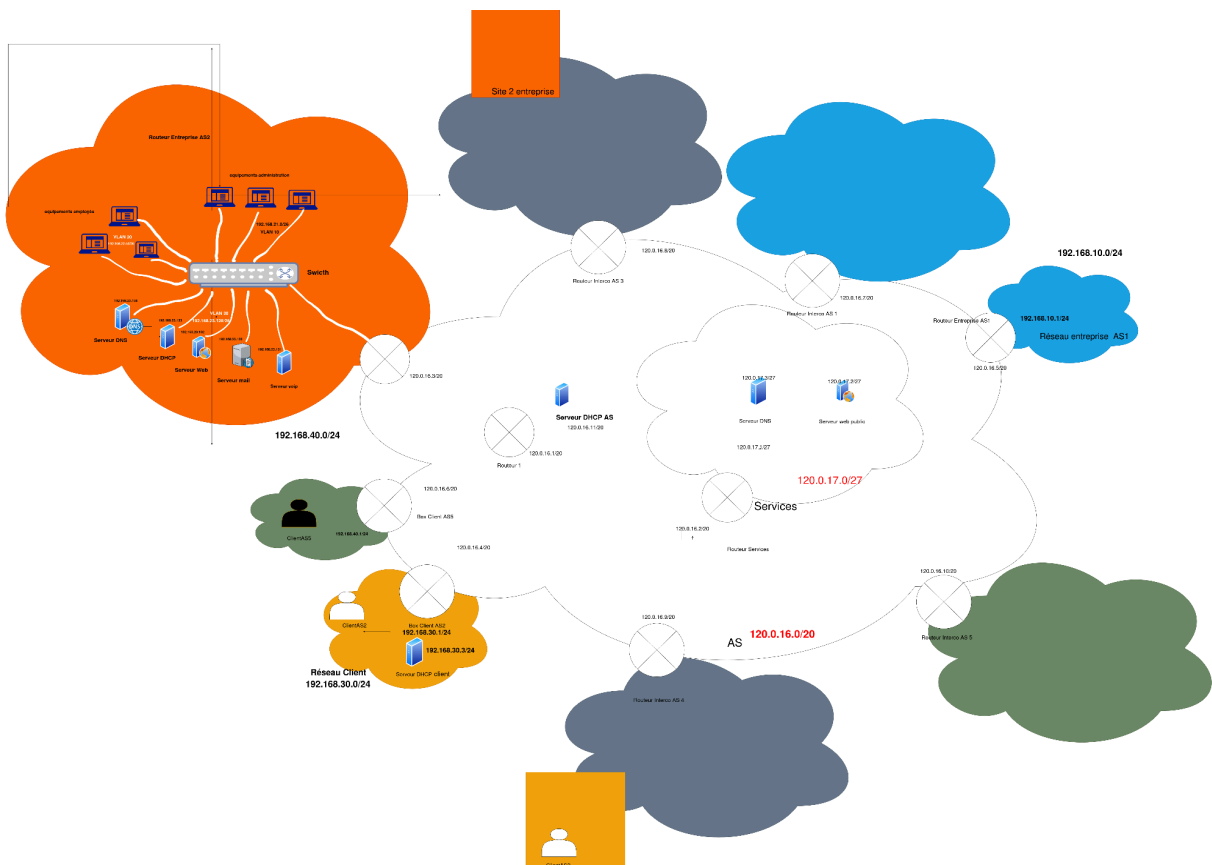
- Un routage dynamique efficace,
- Une gestion des abonnements et résiliations,
- Un service plug and play de type "box-like",
- Une sécurité de l'accès,
- Un accès des particuliers vers le site d'entreprise.

Notre choix d'implémentation virtualisé s'est porté sur Docker, ce logiciel permettant de simuler réellement sous forme de containers un réseau.

Après avoir choisi quelle image utiliser pour chaque machine suivant sa nature (serveur VOIP, routeur, ...), il nous a fallu les configurer une à une.

À des fins de réutilisabilité et de centralisation de configuration, nous avons choisi de créer une arborescence machine que l'on passera aux containers sous formes de volumes persistants, accompagnés de fichier de configuration shell pour l'installation de dépendances, la configuration de nameserver etc.

L'orchestration globale se fait grâce à l'utilisation d'un fichier docker-compose.



A. Serveur SMTP

Nous avons décidé de mettre en place un serveur mail pour permettre aux employés de l'entreprise de s'échanger des mails. L'échange de mails repose sur un modèle client-serveur utilisant deux protocoles distincts pour séparer l'envoi de la réception :

- **SMTP (Simple Mail Transfer Protocol) - Port 25** : C'est le protocole d'envoi. Il est utilisé par le client pour "pousser" le mail vers le serveur, et entre les serveurs pour s'échanger les messages.

Postfix est le moteur de ce protocole. Son rôle est de vérifier si l'expéditeur a le droit d'envoyer un mail, de filtrer les destinations et de déposer le message dans la "boîte aux lettres" physique sur le disque dur du serveur.

- **IMAP (Internet Message Access Protocol) - Port 143** : C'est le protocole de lecture. Il permet au client de se connecter au serveur pour consulter les mails stockés, sans les supprimer du serveur.

Dovecot est l'outil qui gère cette partie. Il s'occupe de l'authentification (vérification du login/mot de passe) et va chercher les fichiers de mails sur le disque pour les présenter de manière organisée au client

Configuration de Postfix (SMTP) pour un Réseau Local d'Entreprise

Étant donné que notre objectif est de mettre en place un service de messagerie interne pour l'entreprise, la configuration de Postfix sera simplifiée car il n'y a pas besoin de gérer le relais vers l'Internet public.

/etc/postfix/main.cf : Le fichier de configuration principal où on définit les règles de base du serveur.

On a :

1. **myhostname** : Nom de l'hôte du serveur de messagerie (doit être résolu par le DNS interne)
2. **mydomain** : Domaine de l'entreprise
3. **myorigin** : domaine qui apparaît par défaut dans les mails sortant
4. **inet_interfaces** : Interfaces d'écoute. Écoutez sur l'interface locale si Postfix est utilisé par d'autres services sur le même hôte
5. **mydestination** : Domaines pour lesquels Postfix accepte la livraison locale (les boîtes aux lettres des employés)

6. **mynetworks** : Le paramètre le plus crucial pour un réseau local. Il définit quels réseaux sont considérés comme "de confiance" et peuvent envoyer des mails sans authentification
7. **home_mailbox** : Indique l'emplacement des boîtes aux lettres locales (Postfix pour déposer les messages, puis Dovecot pour les lire)

Concernant le Dovecot, la configuration se concentre principalement sur l'authentification et l'emplacement des mails.

Fichiers de Configuration Principaux

1. **/etc/dovecot/conf.d/10-auth.conf** : Gère les mécanismes d'authentification.
2. **/etc/dovecot/conf.d/10-mail.conf** : Gère le format et l'emplacement des boîtes aux lettres.
3. **/etc/dovecot/conf.d/10-master.conf** : Gère les services et les ports d'écoute.

Sur docker, nous avons utilisé l'image [ghcr.io/docker-mailserver/docker-mailserver:latest](https://github.com/docker-mailserver/docker-mailserver), contenant tout ce qu'il faut (avec postfix et dovecot déjà préinstallé).

Nous avons uniquement eu à modifier les fichiers nous concernant.

Après avoir lancé le conteneur, on peut alors ajouter des utilisateurs depuis le conteneur serveur-mail avec la commande :
setup email add user_a@entreprise.local motdepasse

Pour envoyer un mail, il faut suivre ces étapes (depuis une station de l'entreprise), en validant après chaque ligne dans l'ordre :

```
telnet 192.168.23.130 25
EHLO pc1.entreprise.local
MAIL FROM:<user_a@entreprise.local>
RCPT TO:<user_b@entreprise.local>
DATA
Subject: Test de fonctionnement
Ceci est le corps du message
. (ce point est important)
QUIT
```

Et pour lire un mail reçu depuis un terminal, il nous suffit d'exécuter :

```
curl "imap://192.168.23.130/INBOX;UID=1" --user
"user_b@entreprise.local:motdepasse"
```

B. Vlan dans l'entreprise

Pour notre entreprise, nous avons mis en place des vlans pour mieux organiser et sécuriser le réseau. L'idée est simple : au lieu que tout le monde soit mélangé, créer des groupes séparés pour que les flux ne se parasitent pas et pour contrôler qui accède à quoi. Concrètement, nous avons créé un Vlan 10 pour isoler les équipements de l'administration, un Vlan 20 pour les employés, et un Vlan 30 pour protéger les serveurs qui hébergent les services comme le DNS, DHCP, VoIP, Mail, le Web.

Pour configurer le switch par rapport à cette architecture, il faut deux types de ports : les **ports Access** et le **port Trunk**. Les ports Access servent à connecter les équipements terminaux (PC de l'administration, PC des employés) et les serveurs en les isolant dans leurs VLANs respectifs (10, 20 ou 30). Le port Trunk, quant à lui, assure la connexion montante vers le routeur et permet de transporter les flux de tous les VLANs sur un seul lien physique grâce au taggage des trames. Ce dernier va permettre la communication entre les VLANs via le routage inter-VLAN .

Nous avons voulu que le serveur DHCP puisse distribuer dynamiquement les adresses IP des équipements dans les VLAN 10 et 20. Pour ce faire, le routeur joue un rôle de passerelle et de relais :

1. Puisque le serveur DHCP est situé dans le VLAN 30, les requêtes broadcast des clients des VLAN 10 et 20 ne peuvent pas l'atteindre directement. Nous avons donc configuré une fonction de **relais DHCP** (agent de relais) sur chaque sous-interface du routeur.
2. Lorsqu'un équipement du VLAN 10 ou 20 envoie une demande d'adresse, le routeur intercepte ce message et le réexpédie directement vers l'adresse IP du serveur DHCP dans le VLAN 30
3. Le serveur DHCP identifie la provenance de la requête grâce à l'adresse de la sous-interface du routeur et renvoie une configuration IP

Puisque Docker ne propose pas de conteneur switch natif permettant de configurer des ports en mode Access ou Trunk comme sur du matériel réel, nous avons adapté notre simulation pour respecter les objectifs du projet. Pour représenter la segmentation de l'entreprise, nous avons découpé le réseau principal en plusieurs sous-réseaux logiques. Cette approche nous a permis de reproduire l'idée et le fonctionnement des **VLAN 10, 20 et 30**, en isolant les flux de l'administration, des employés et des serveurs au niveau de l'adressage IP.

C. VPN entre l'entreprise et les particuliers

Afin de permettre un accès aux ressources de l'entreprise depuis l'extérieur, pour des utilisateurs autorisés, nous avons utilisé le VPN WireGuard (WG) qui permet un chiffrement sécurisé.

La sécurité se fait à l'aide de clés privées et publiques, que l'on peut générer via la ligne suivante :

```
wg genkey | tee /dev/tty | wg pubkey
```

Ainsi, en générant 2 paires de clés, une pour l'entreprise et une pour le client, il est possible de créer un accès VPN chiffré entre les 2, et permettre à l'employé d'avoir accès à toutes les ressources de l'entreprise depuis n'importe où.

L'utilisation des 2 clés se fait de la manière suivante :

- La clé privée de l'entreprise est écrite dans un document, au niveau de l'entreprise. (Symétrique du côté de l'utilisateur)
- La clé publique de l'entreprise est donnée à l'utilisateur, afin de vérifier si la connexion lui est autorisée. (Symétrique du côté de l'utilisateur)

Pour créer ce tunnel entre l'entreprise et l'utilisateur, il faut également forcer l'écoute sur le port 51820, qui est le port standard d'écoute de WireGuard, sinon le port d'écoute sera pris de manière aléatoire et ne permettra pas d'établir la liaison.

Pour s'assurer que le VPN est bien utilisé, il est possible de faire un traceroute ip_web, qui nous permettrait alors de voir que l'on passe par le tunnel pour accéder au site web privé.

En utilisant la commande traceroute, le terminal nous renvoie:

```
/ # traceroute 192.168.23.132
traceroute to 192.168.23.132 (192.168.23.132), 30 hops max, 46 byte packets
 1 10.0.0.1 (10.0.0.1) 0.490 ms 0.485 ms 0.348 ms
 2 192.168.23.132 (192.168.23.132) 0.300 ms 0.239 ms 0.297 ms
/ #
```

La ligne 1 (10.0.0.1) confirme que l'on passe bien par le VPN pour accéder au site web de l'entreprise.

D. Les serveurs Web (entreprise et public)

Pour les serveurs Web, il faut juste déposer les fichiers html les images etc dans le répertoire racine du serveur (souvent /var/www/html).

E. Les serveurs DNS (entreprise et public)

Afin d'assurer la résolution des noms de domaine, l'accès aux sites Web, et la résolution de noms d'hôtes, la mise en place d'un serveur DNS est indispensable.

Toutes les configurations se passent dans les fichiers :

1. **named.conf.local**, où on définit les zones que couvre le serveur DNS.
2. [db.sitepublic.com](#) ou [db.sitentreprise.com](#), où l'on crée réellement une correspondance entre nom et adresse IP.
3. **named.conf.options**, où l'on indique dans ce fichier quel autre serveur DNS sera interrogé dans le cas où la résolution du nom de domaine n'aurait pas abouti.

F. DHCP client / Accès de type box-like

Afin d'assurer l'adressage dynamique des différents clients et services de l'entreprise et permettre un accès de type "box-like" il est nécessaire d'implémenter une solution DHCP pour les clients également.

Pour cela, on utilise l'image docker **networkboot/dhcpd**.

Enfin il ne reste plus qu'à écrire un fichier de config : **dhcpd.conf**. Ce dernier renseigne la plage d'adresse que l'on veut attribuer mais aussi la passerelle par défaut et quel serveur DNS sélectionner.

Pour tester le bon fonctionnement de DHCP, il nous a simplement suffit de vérifier que chaque équipement du client se voyait bien attribuer une adresse ip dans la range définie précédemment.

G. Serveur VOIP

Pour permettre la communication vocale entre les différentes personnes de l'entreprise de notre AS, nous avons configuré un serveur Asterisk.

Pour cela, nous avons sélectionné l'image adéquate sur docker : **Andrius/Asterisk**

En ce qui concerne les fichiers de configuration ils sont au nombre de deux : **pjsip.conf** et **extensions.conf**.

Le premier fichier permet la gestion des utilisateurs. En effet, on définit dans ce fichier les deux utilisateurs entrant en communication (ici nommés 1001 et 1002,) ainsi que les mots de passe permettant d'établir la communication. Le mode de transport est aussi rappelé dans le fichier : UDP dans notre cas, pour favoriser la rapidité (essentielle lors d'une communication voix).

Le deuxième fichier définit le plan de numérotation et ce que doit faire le serveur quand un numéro est composé. Par exemple, faire sonner un appareil pendant une durée de 20 secondes, couper la ligne si l'appel n'est pas décroché au bout de ce même temps, etc.

Pour tester le serveur, nous utiliserons **PJSUA** qui permet de simuler un terminal VoIP, de s'enregistrer sur le serveur Asterisk et d'émettre des appels.

Pour s'assurer du bon fonctionnement de Voip, on simule un appel entre des personnes de l'entreprise à l'aide de **pjsua**. Pour cela, il nous faut du côté du destinataire effectuer cette commande (l'adresse décrite étant celle du serveur voip) :

```
pjsua --id sip:1002@192.168.23.131 \  
      --registrar sip:192.168.23.131 \  
      --realm asterisk \  
      --user 1002 \  
      --password password1002 \  
      --local-port 5060 \  
      --null-audio
```

et du côté de l'émetteur :

```
pjsua --id sip:1001@192.168.23.131 \  
      --registrar sip:192.168.23.131 \  
      --realm asterisk \  
      --user 1001 \  
      --password password1001 \  
      --null-audio \  
      sip:1002@192.168.23.131
```

Ensuite sur le terminal du destinataire on peut lire que l'appel entre :

```
11:11:11.469      pjsua_aud.c  ..Conf connect: 2 --> 0
11:11:11.469      pjsua_aud.c  ...Set sound device: capture=-99, playb
ack=-99, mode=0, use_default_settings=0
11:11:11.469      pjsua_aud.c  ....Null sound device, mode setting is
ignored
11:11:11.469      pjsua_aud.c  ....Setting null sound device..
11:11:11.469      pjsua_app.c  ....Turning sound device -99 -99 ON
11:11:11.469      pjsua_aud.c  ....Opening null sound device..
11:11:11.469      conference.c  ....Connect ports 2->0 queued
11:11:11.469      pjsua_app.c  ..Incoming call for account 0!
Media count: 1 audio & 0 video & 0 text
From: <sip:1001@192.168.20.8>
To: <sip:1002@192.168.20.10;ob>
Press a to answer or h to reject call
11:11:11.489      conference.c  !.Port 2 (ring) transmitting to port 0 (
Master/sound)
a
Answer with code (100-699) (empty to cancel):
```

Il suffit d'appuyer sur la touche **a** pour décrocher puis on peut s'assurer qu'on a un appel en cours en appuyant sur la touche **entrée** :

```
+=====+
==+
You have 1 active call
Current call id=0 to <sip:1001@192.168.20.8> [INCOMING]
>>>
```

H. Routage dynamique par OSPF

Afin de rendre l'infrastructure plus flexible et d'automatiser la découverte des réseaux (notamment les nouveaux VLANs et autres AS), nous avons implémenté le protocole de routage dynamique OSPF (Open Shortest Path First). Cette étape a permis de remplacer l'intégralité du routage statique par une gestion dynamique.

1. Configuration de FRRouting (FRR)

Pour chaque routeur de la topologie, nous avons intégré la suite logicielle FRR. La configuration repose sur l'activation des démons zebra et ospfd via le fichier daemons, et la définition des annonces réseau dans frr.conf.

Aperçu de la configuration OSPF (Exemple : routeur-services) :

```
router ospf
ospf router-id 2.2.2.2
! Annonce du réseau d'interconnexion entre les AS
network 120.0.16.0/24 area 0
! suite ...
```

2. Automatisation et Sécurité (iptables)

Le passage au routage dynamique a nécessité une adaptation des scripts d'initialisation pour garantir la cohabitation entre le routage et la sécurité. Nous avons autorisé explicitement le protocole OSPF dans **iptables** pour permettre la découverte des zones voisines.

Aperçu des commandes d'initialisation (Exemple : routeur-services) :

```
# Autorisation du protocole OSPF dans le pare-feu
iptables -A INPUT -p ospf -j ACCEPT
iptables -A FORWARD -p ospf -j ACCEPT

# Gestion des permissions et lancement de FRR
chown -R frr:frr /etc/frr && chmod -R 640 /etc/frr/daemons && exec
/usr/lib/frr/docker-start
```

3. Cohabitation OSPF et VPN

Un point critique de l'implémentation a été de faire cohabiter OSPF avec le tunnel WireGuard. Bien que OSPF apprenne dynamiquement les routes vers les réseaux privés, nous avons maintenu des routes statiques spécifiques vers l'interface wg0.

Cette approche possède alors 2 avantages majeurs :

- 1. Une résilience de l'infrastructure** : OSPF assure la connectivité entre les adresses publiques des routeurs. Si un lien physique tombe, OSPF recalcule instantanément un nouveau chemin, permettant au tunnel VPN de rester actif sans interruption.
- 2. Une meilleure confidentialité des données** : En forçant les plages d'IP privées vers wg0 après la convergence OSPF, nous garantissons que le trafic sensible est systématiquement chiffré, même si OSPF propose un chemin plus court via le réseau public.

Aperçu de la gestion de priorité (Exemple : routeur-client-as-2) :

```
# Lancement d'OSPF puis temporisation pour la convergence
/usr/lib/frr/docker-start &
sleep 5

# Forçage du trafic privé dans le tunnel (Priorité sur OSPF)
ip route add 192.168.23.128/26 dev wg0
```

I. DHCP au niveau de l'AS (accès box-like et gestion des souscriptions)

Au niveau de l'AS, le service DHCP remplit deux objectifs principaux

1. **Fournir un accès Internet de type box-like** aux clients particuliers, entièrement automatique et sans configuration manuelle.
2. **Assurer la gestion des souscriptions et des résiliations**, en contrôlant l'accès au réseau de l'AS.

D'une part, l'AS doit proposer une solution d'interconnexion sans configuration pour ses clients. Cette exigence vise à fournir un accès Internet de type **box-like**, c'est-à-dire automatique et transparent, sans intervention technique du client.

Pour répondre à cette contrainte, un service DHCP a été déployé au niveau de l'AS afin d'assurer :

- l'attribution dynamique des adresses IP.
- la fourniture automatique des paramètres réseau (passerelle par défaut et serveur DNS).
- le contrôle d'accès au réseau de l'AS.

D'autre part, dans une logique de fournisseur d'accès, **une gestion des souscriptions et des résiliations** a été implémentée par un mécanisme de contrôle basé sur **les adresses MAC des box clients**.

Seules les box dont l'adresse MAC est **enregistrée dans la configuration du serveur DHCP** sont autorisées à obtenir une adresse IP. Les équipements non reconnus sont automatiquement rejetés.

Cette approche permet de simuler un mécanisme réaliste d'activation et de désactivation des abonnements tout en conservant une attribution dynamique des adresses IP et une interconnexion entièrement automatique pour les particuliers.

La sécurité et le contrôle d'accès se font via la directive **deny unknown-clients** dans le fichier de configuration dhcpd.conf. Seules les adresses MAC explicitement déclarées dans la configuration peuvent obtenir une adresse IP et accéder au réseau de l'AS.

Extrait du fichier dhcpd.conf :

```
deny unknown-clients;
subnet 120.0.16.0 netmask 255.255.255.0 {
    range 120.0.16.50 120.0.16.200;
    option routers 120.0.16.1;
    option domain-name-servers 120.0.17.3;
}
host client1 {
    hardware ethernet 02:42:ac:11:00:01;
}
host client2 {
    hardware ethernet 02:42:ac:11:00:02;
}
```

J. DNS split horizon au niveau des box clients

Afin de garantir un accès cohérent et sécurisé aux services de l'entreprise pour les clients particuliers connectés via VPN, ces derniers doivent pouvoir accéder aux ressources internes de l'entreprise, notamment au domaine **entreprise.com**.

Dans une première approche, la box du particulier utilisait exclusivement le **DNS de l'entreprise** comme serveur DNS par défaut, celui-ci effectuant ensuite un transfert des requêtes vers le DNS de l'AS pour les domaines publics.

Cette solution présente plusieurs limitations :

- la surcharge inutile du serveur DNS de l'entreprise.
- l'augmentation du trafic dans le tunnel VPN particulier-entreprise.
- la dégradation potentielle des performances de résolution pour les domaines publics.

Pour éliminer ce trafic superflu et optimiser l'utilisation des ressources réseau, une architecture **DNS split horizon** a été implémentée au niveau de la box client à l'aide de **dnsmasq**.

Cette solution permet de réduire la charge sur l'infrastructure de l'entreprise, d'optimiser le trafic VPN et d'améliorer l'efficacité globale de l'architecture réseau.

Le fonctionnement du DNS split-horizon se fait de la manière suivante :

- Les requêtes DNS avec le suffixe entreprise.com sont dirigées vers le serveur DNS de l'entreprise (192.168.23.134)

- Toutes les autres requêtes DNS sont envoyées directement vers le serveur DNS de l'AS (120.0.17.3)
- Le **service dnsmasq** écoute localement (127.0.0.1) ainsi que sur l'interface LAN de la box (192.168.30.1)

Fichier dnsmasq.conf (dans la box particulier) :

```
# Serveur DNS par défaut
server=120.0.17.3
# Serveur DNS pour entreprise.com
server=/entreprise.com/192.168.23.134
# Ecoute sur ces interfaces
listen-address=127.0.0.1,192.168.30.1
```

K. Mise en place des réseaux privés et du firewall

Afin de mettre en place des réseaux privés, il nous est nécessaire de changer les iptables et l'adressage afin de prendre en compte le fait que les adresses clients soient non routables.

De plus, nous souhaitons que ces réseaux soient sécurisés et ne permettent d'entrer aucun paquet si ce n'est en réponse à des requêtes préalablement établies, et que toute requête sortante puisse passer et être routé (donc via un NAT/PAT fait par la box).

Ces spécifications nous ont alors menées à rédiger les fichiers de configuration des box clients tel que suis :

```
# Rien ne passe.
iptables -P FORWARD DROP
# Sauf les connexions déjà établies.
iptables -A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT
# Et les requêtes sortant du réseau privé.
iptables -A FORWARD -i eth1 -o eth0 -j ACCEPT
# On remplace l'adresse ip privée des machines par l'adresse publique du
routeur
iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
```

2. Répartitions des tâches :

Noms	Tâches
MEDIANI Hocine	Architecture globale, firewall (réseau privé et sécurité d'accès), DNS d'AS
KRIEF Benjamin	Implémentation de VPN par Wireguard, DHCP chez le client et architecture
MBAITODJIMREOU ELDJIMBAYE Bonheur	DNS entreprise, serveur mail, vlan dans l'entreprise, DHCP de l'entreprise
BERRAD Marouane	DHCP AS (accès box-like et gestion des souscriptions), DNS split horizon pour les clients particuliers
NADI Aymen	Routage dynamique par OSPF, adaptation partout dans l'architecture.
KRAIFI Ilian	Implémentation routeur de type "box-like" DHCP côté client et VOIP par Asterisk